Проектирование электронной системы сдачи в аренду торговых площадей

1. Введение

Проектируется веб-система «АрендаТЦ», которая переводит в электронный вид весь цикл аренды торговых площадей: от поиска свободного помещения до начисления и учёта арендной платы.

Цель работы — спроектировать информационную систему, сокращающую срок заключения договора аренды и исключающую ручной учёт площадей, заявок и платежей.

Задачи:

1. Проанализировать предметную область и бизнес-процессы аренды торговых площадей.
2. Построить функциональную модель IDEF0 и диаграмму потоков данных DFD.
3. Сформулировать функциональные и нефункциональные требования.
4. Разработать UML-модели: варианты использования, деятельность, последовательность, классы, состояния.
5. Спроектировать ER-модель и физическую схему базы данных.
6. Определить архитектуру, технологический стек и основные экраны интерфейса.

Объект проектирования — процесс сдачи в аренду помещений торгового центра управляющей компанией.

Предмет проектирования — электронная система, автоматизирующая учёт площадей, приём заявок, оформление договоров и расчёты с арендаторами.

2. Анализ предметной области

Управляющая компания торгового центра сдаёт помещения арендаторам — розничным сетям, кафе, сервисным компаниям. Сейчас площади ведутся в таблицах Excel, заявки приходят по почте и телефону, договоры готовятся в Word, а начисления сверяются вручную.

2.1. Участники процесса

Участник	Роль в процессе
Потенциальный арендатор	Ищет помещение, подаёт заявку, согласует условия

Участник	Роль в процессе
Арендатор	Подписывает договор, получает счета, оплачивает, подаёт обращения
Менеджер по аренде	Ведёт каталог площадей, рассматривает заявки, готовит договоры
Бухгалтер	Выставляет счета, контролирует оплату и задолженность
Руководитель УК	Утверждает договоры и тарифы, смотрит отчёты по заполняемости
Администратор	Управляет пользователями, ролями и справочниками

2.2. Основные бизнес-процессы

1. **Ведение каталога площадей** — учёт помещений: этаж, площадь, тип, ставка за м², состояние.
2. **Приём и рассмотрение заявок** — проверка арендатора, согласование срока и условий.
3. **Бронирование помещения** — временная блокировка площади на период согласования.
4. **Оформление договора** — формирование по шаблону, согласование, утверждение и подписание.
5. **Начисление и учёт оплаты** — ежемесячные счета за аренду и эксплуатационные услуги, пени за просрочку.
6. **Сопровождение договора** — обращения арендатора, продление, расторжение, возврат помещения.

2.3. Проблемы текущего процесса

- Одна площадь может быть обещана двум арендаторам: нет единого актуального статуса помещения.
- Заявки теряются между почтой, телефоном и мессенджерами; нет истории согласования.
- Договор собирается вручную, отсюда ошибки в ставках, площадях и сроках.
- Задолженность выявляется с опозданием, потому что оплаты сверяются с выписками вручную.
- Руководство не видит заполняемость и доходность площадей в реальном времени.

2.4. Обзор аналогов

Решение	Сильные стороны	Ограничения для задачи
1С:Управление недвижимостью	Бухгалтерский учёт, начисления, отчётность	Нет самообслуживания арендатора, тяжёлое внедрение
Yardi / MRI (зарубежные)	Полный цикл коммерческой недвижимости	Высокая стоимость, ограниченная доступность в РФ

Решение	Сильные стороны	Ограничения для задачи
Доски объявлений (Циан, Авито)	Широкий охват арендаторов	Только поиск; договоры и расчёты не покрывают

Вывод: нужна система с публичным каталогом и личным кабинетом арендатора, связанная с учётом договоров и начислений, с возможностью выгрузки данных в бухгалтерию (1С).

3. Функциональная модель IDEF0

Верхний уровень системы — одна функция A0 «Сдавать в аренду торговые площади»; её декомпозиция даёт пять подфункций, последовательно передающих друг другу помещения, заявки, договоры и счета.

Точка зрения: руководитель управляющей компании. **Цель модели:** показать, как заявки арендаторов превращаются в действующие договоры и поступления денег.

3.1. Контекстная диаграмма A-0

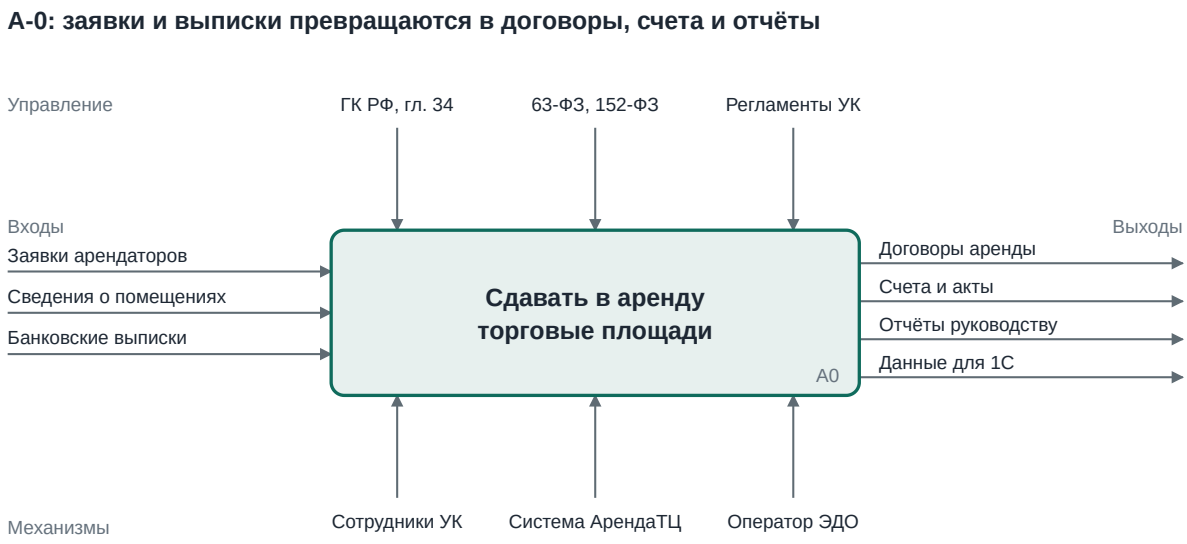


Рисунок 1 — Контекстная диаграмма A-0

Стрелки слева — входы, справа — выходы, сверху — управление (законы и регламенты), снизу — механизмы (кто и чем выполняет функцию).

3.2. Декомпозиция A0

Функция A0 раскладывается на пять блоков; выход каждого служит входом или управлением для следующего, а начисления (A4) возвращают данные о задолженности в сопровождение (A5).

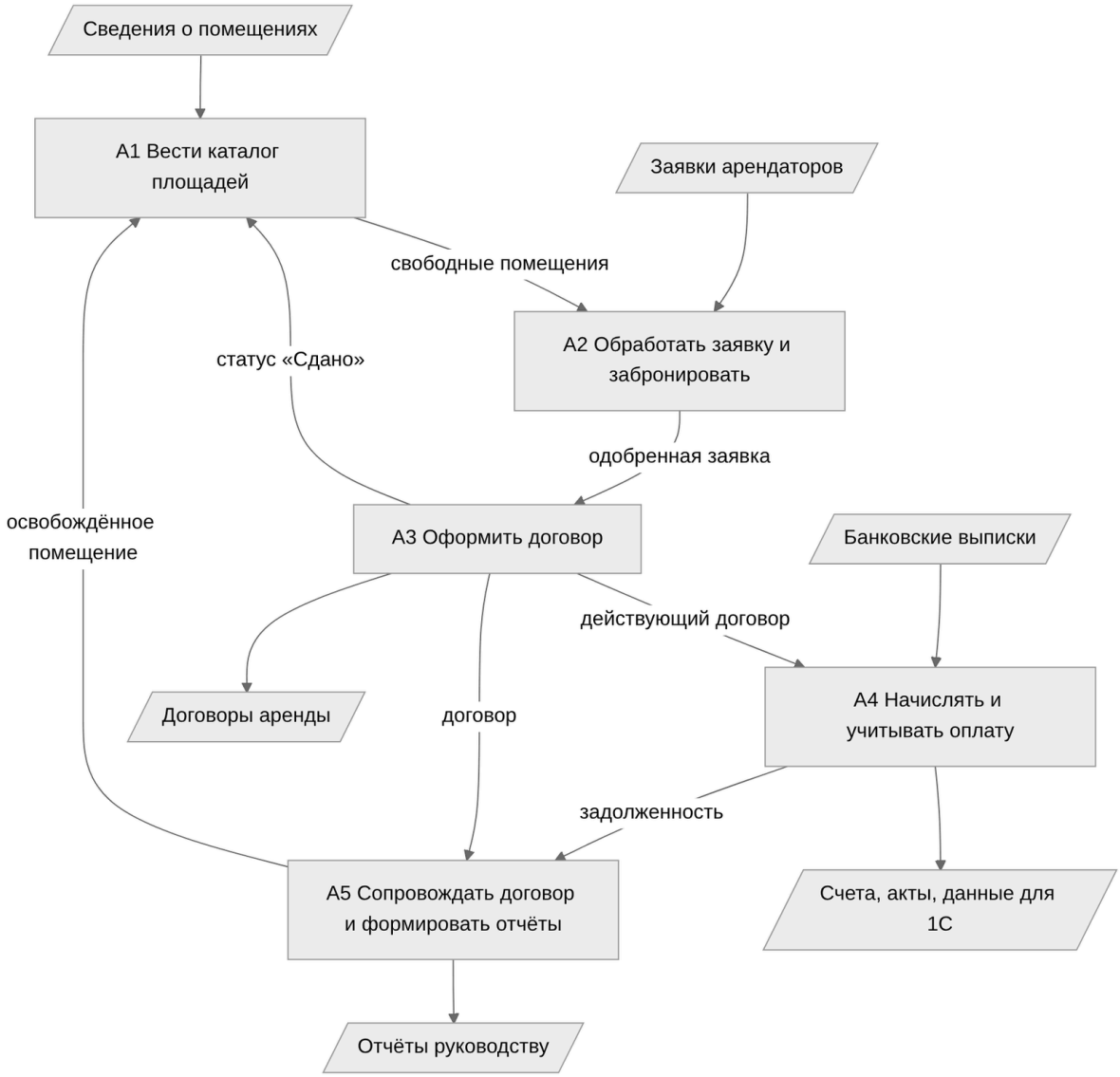


Рисунок 2 — Декомпозиция функции A0 на пять блоков

Блок	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A1 Вести каталог площадей	Сведения о помещениях, освобождённые помещения	Регламенты УК, тарифы	Каталог свободных помещений	Менеджер, система
A2 Обработать заявку и забронировать	Заявка арендатора, каталог	Регламент рассмотрения заявок, 152-ФЗ	Одобренная заявка, бронь	Менеджер, API ФНС, система
A3 Оформить договор	Одобренная заявка, реквизиты арендатора	ГК РФ гл. 34, 63-ФЗ, шаблон договора	Подписанный договор	Менеджер, юрист, руководитель, оператор ЭДО

Блок	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A4 Начислять и учитывать оплату	Договор, банковская выписка	Условия договора, учётная политика	Счета, акты, задолженность, выгрузка в 1С	Бухгалтер, система
A5 Сопровождать договор и формировать отчёты	Договор, задолженность, обращения	Регламенты УК	Отчёты, допсоглашения, возврат помещения	Менеджер, руководитель, система

Для отчёта диаграмму A0 удобно построить в Ramus или Business Studio в классической «лестничной» раскладке, используя эту таблицу как спецификацию стрелок.

4. Диаграмма потоков данных (DFD)

DFD первого уровня в нотации Гейна — Сарсона показывает пять процессов, четыре внешние сущности и шесть хранилищ данных; все финансовые потоки проходят через хранилище «Счета».

Обозначения: прямоугольник — внешняя сущность, скруглённый блок — процесс, блок с номером D — хранилище данных.



Рисунок 3 — Диаграмма потоков данных первого уровня

Менеджер по аренде и бухгалтер на DFD не показаны отдельно: они — пользователи процессов 1–4 внутри системы, а не внешние источники данных.

5. Требования к системе

Система обслуживает шесть ролей и должна обеспечить 24 функции, сгруппированные по подсистемам; ключевое требование — исключить двойное бронирование одной площади.

5.1. Роли и права доступа

Функция	Гость	Арендатор	Менеджер	Бухгалтер	Руководитель	Админ
Просмотр каталога и плана этажей	+	+	+	+	+	+
Подача заявки, бронирование		+				
Рассмотрение заявок, ведение площадей			+			
Формирование договора			+			
Утверждение договора и тарифов					+	
Начисления, счета, оплаты		просмотр		+	просмотр	
Обращения арендатора	—	создание	обработка			
Отчёты			+	+	+	
Пользователи, справочники						+

5.2. Функциональные требования

Каталог площадей

- ФТ-1. Хранить помещения с атрибутами: номер, этаж, площадь (м²), тип (торговое, общепит, склад, островное), ставка (руб./м² в месяц), статус.
- ФТ-2. Показывать свободные помещения на интерактивном плане этажа.
- ФТ-3. Фильтровать по этажу, площади, типу, цене и дате освобождения.

Заявки и бронирование

- ФТ-4. Регистрировать арендатора (юрлицо или ИП) с проверкой ИНН/ОГРН.
- ФТ-5. Принимать заявку на помещение с желаемым сроком и видом деятельности.
- ФТ-6. Бронировать помещение на заданный срок (по умолчанию 5 рабочих дней), блокируя его для других заявок.
- ФТ-7. Автоматически снимать бронь по истечении срока.
- ФТ-8. Уведомлять участников о смене статуса заявки (email, личный кабинет).

Договоры

- ФТ-9. Формировать договор по шаблону с подстановкой реквизитов, площади, ставки и срока.
- ФТ-10. Вести маршрут согласования: менеджер → юрист → руководитель.
- ФТ-11. Подписывать договор усиленной квалифицированной электронной подписью (УКЭП).
- ФТ-12. Хранить версии договора и дополнительные соглашения.
- ФТ-13. Напоминать о сроке окончания договора за 60 дней; оформлять продление и расторжение.

Финансы

- ФТ-14. Ежемесячно начислять постоянную часть аренды и переменную (эксплуатация, коммунальные услуги).
- ФТ-15. Формировать счета и акты, отправлять их арендатору.
- ФТ-16. Загружать банковские выписки и сопоставлять платежи со счетами.
- ФТ-17. Начислять пени за просрочку по ставке из договора.
- ФТ-18. Учитывать обеспечительный платёж.
- ФТ-19. Выгружать начисления и оплаты в 1С (формат CommerceML/XML).

Сопровождение и отчётность

- ФТ-20. Принимать обращения арендатора (ремонт, доступ, претензии) и отслеживать их статус.
- ФТ-21. Отчёт по заполняемости: доля сданной площади по этажам.
- ФТ-22. Отчёт по доходности и задолженности по арендаторам.
- ФТ-23. Журнал действий пользователей.
- ФТ-24. Управление пользователями, ролями и справочниками.

5.3. Нефункциональные требования

Категория	Требование
Производительность	Ответ на типовой запрос — не более 2 с при 200 одновременных пользователях
Надёжность	Доступность 99,5% в месяц; ежедневное резервное копирование БД
Целостность	Помещение не может иметь двух активных броней или договоров на пересекающиеся периоды (ограничение на уровне БД)
Безопасность	HTTPS, хранение паролей в виде хеша (bcrypt), ролевая модель доступа, защита от OWASP Top 10
Персональные данные	Соответствие 152-ФЗ; хранение данных на серверах в РФ

Категория	Требование
Юридическая значимость	Подписание по 63-ФЗ «Об электронной подписи»
Удобство	Адаптивный интерфейс для ПК и смартфонов; русский язык
Сопровождаемость	Модульная архитектура, REST API с документацией OpenAPI

6. Диаграмма вариантов использования

Шесть актёров работают с 16 прецедентами; общий для всех прецедент «Авторизоваться» подключается через «include».

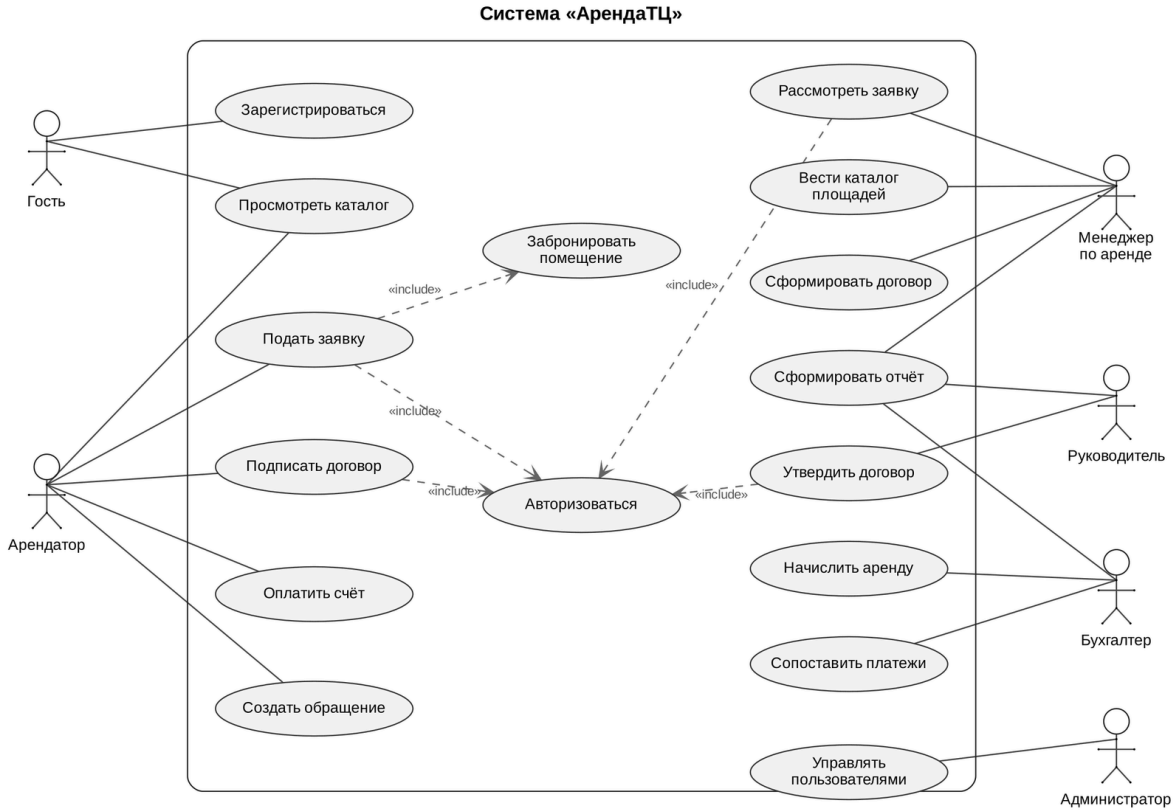


Рисунок 4 — Диаграмма вариантов использования

Актёры-клиенты расположены слева, сотрудники управляющей компании — справа. Пунктирные стрелки «include» показывают обязательно включаемые прецеденты.

6.1. Спецификация прецедента «Подать заявку»

Элемент	Описание
Актёр	Арендатор
Предусловие	Арендатор авторизован, его организация прошла проверку ИНН

Элемент	Описание
Основной поток	1. Арендатор выбирает свободное помещение в каталоге. 2. Указывает срок аренды и вид деятельности. 3. Система проверяет, что помещение свободно на весь период. 4. Система создаёт заявку и бронь на 5 рабочих дней. 5. Менеджер получает уведомление.
Альтернативный поток	3а. Помещение занято на часть периода — система предлагает ближайшую свободную дату или похожие помещения.
Постусловие	Заявка в статусе «На рассмотрении», помещение в статусе «Забронировано»

6.2. Спецификация прецедента «Утвердить договор»

Элемент	Описание
Актёр	Руководитель
Предусловие	Договор сформирован менеджером и согласован юристом
Основной поток	1. Руководитель открывает договор на утверждение. 2. Проверяет ставку, срок и обеспечительный платёж. 3. Подписывает договор УКЭП. 4. Система отправляет договор арендатору на подпись.
Альтернативный поток	2а. Руководитель возвращает договор на доработку с комментарием.
Постусловие	Договор в статусе «Ожидает подписи арендатора»

7. Диаграммы деятельности и последовательности

От заявки до действующего договора проходит четыре точки решения; бронь блокирует помещение на всё время согласования и снимается автоматически при отказе или истечении срока.

7.1. Диаграмма деятельности «Заключение договора аренды»

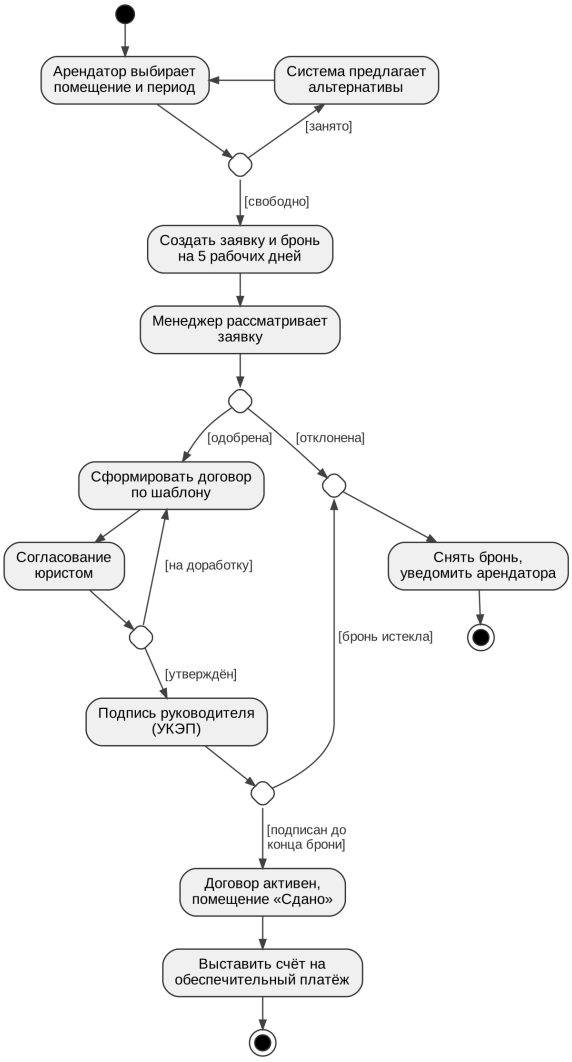


Рисунок 5 — Диаграмма деятельности «Заключение договора аренды»

7.2. Диаграмма последовательности «Подача заявки»

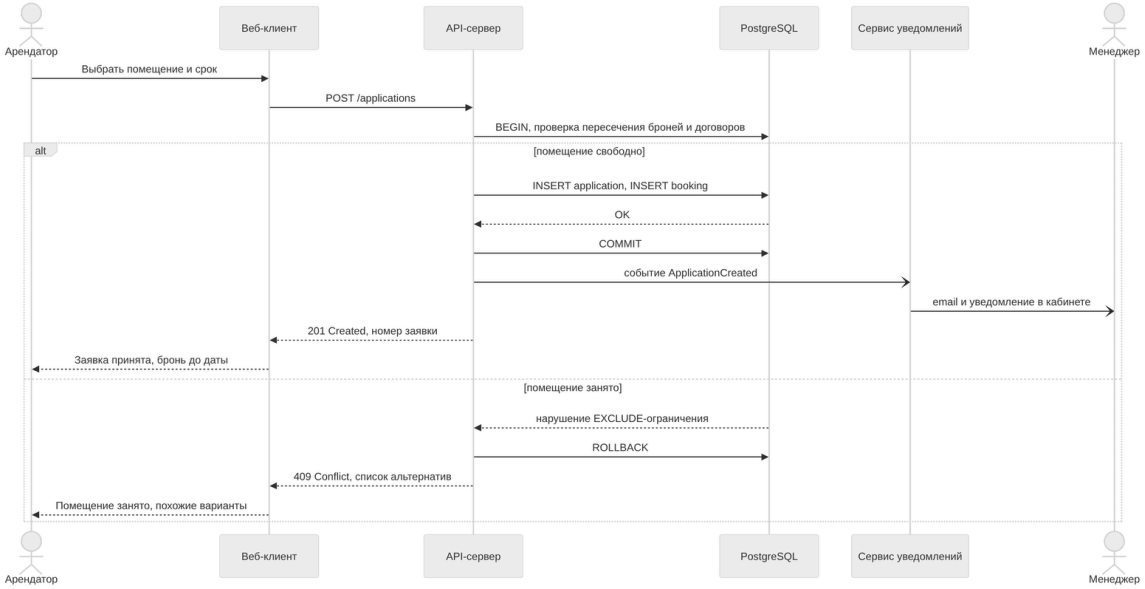


Рисунок 6 — Диаграмма последовательности «Подача заявки»

7.3. Диаграмма последовательности «Сопоставление платежа»

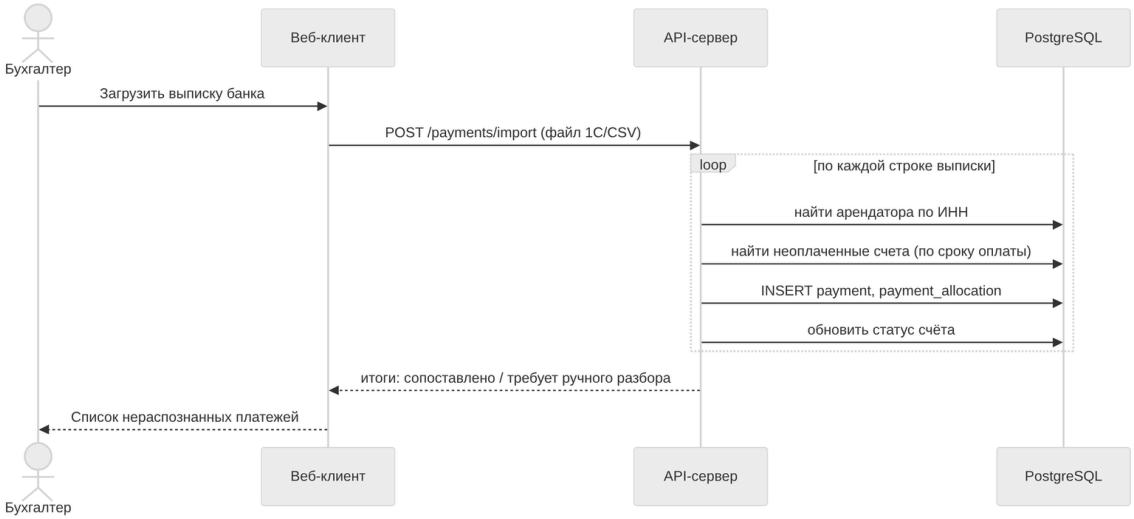


Рисунок 7 — Диаграмма последовательности «Сопоставление платежа»

8. Диаграмма классов и диаграмма состояний

Доменная модель строится вокруг класса Contract: он связывает арендатора с помещением и порождает ежемесячные счета; статусы заявки и договора меняются только через методы классов.

8.1. Диаграмма классов предметной области

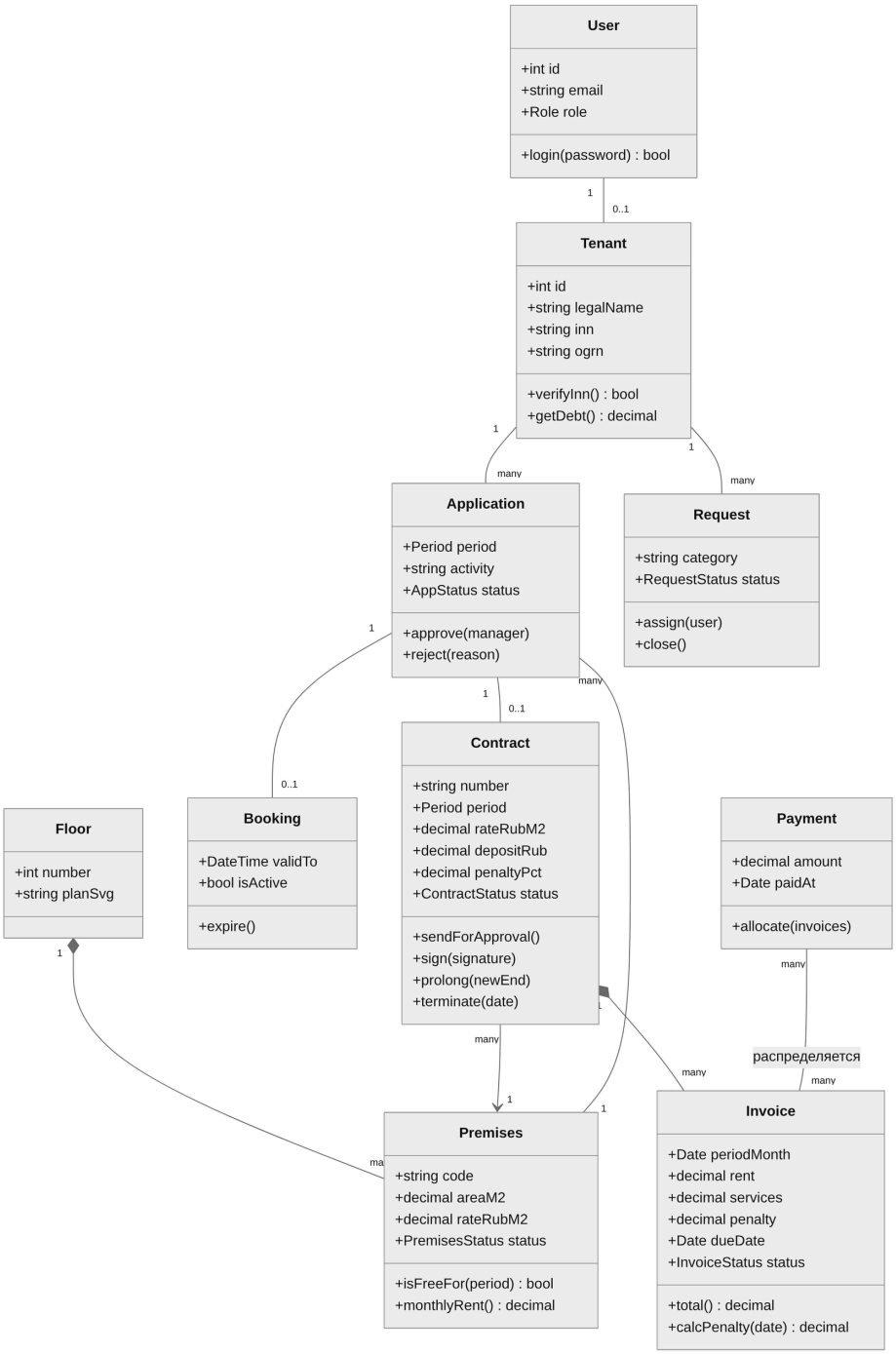


Рисунок 8 — Диаграмма классов предметной области

8.2. Диаграмма состояний договора

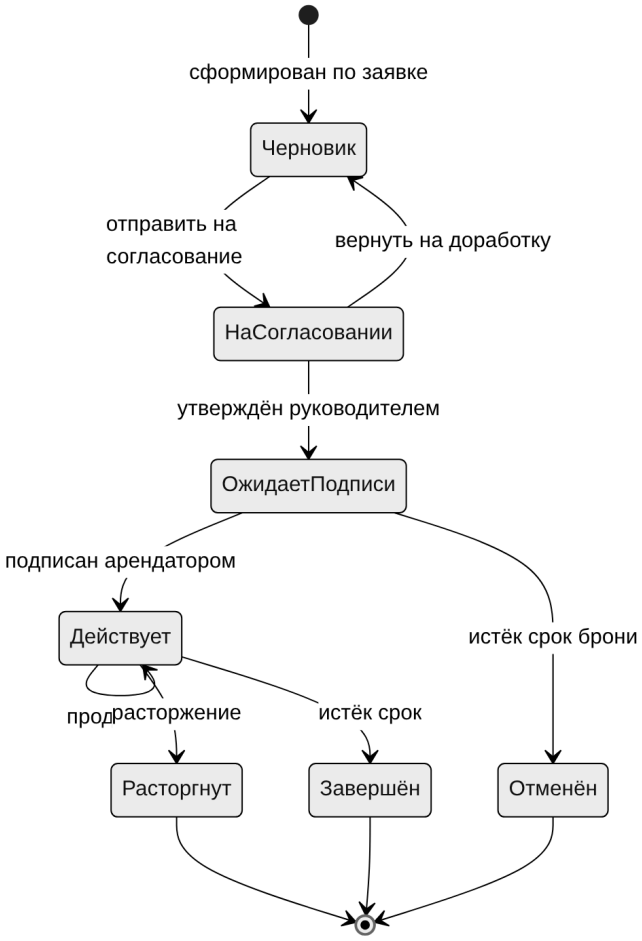


Рисунок 9 — Диаграмма состояний договора

8.3. Диаграмма состояний помещения

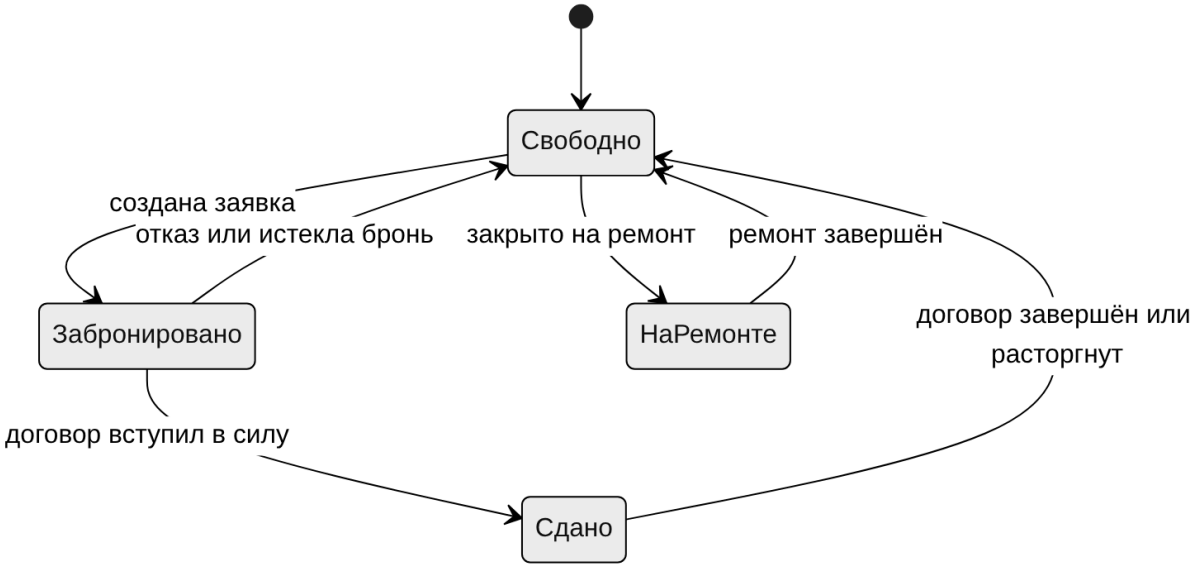


Рисунок 10 — Диаграмма состояний помещения

9. ER-модель и схема базы данных

База данных содержит 14 таблиц в третьей нормальной форме; центральная связка — «Помещение — Заявка — Договор — Начисление — Платёж».

9.1. Логическая ER-модель

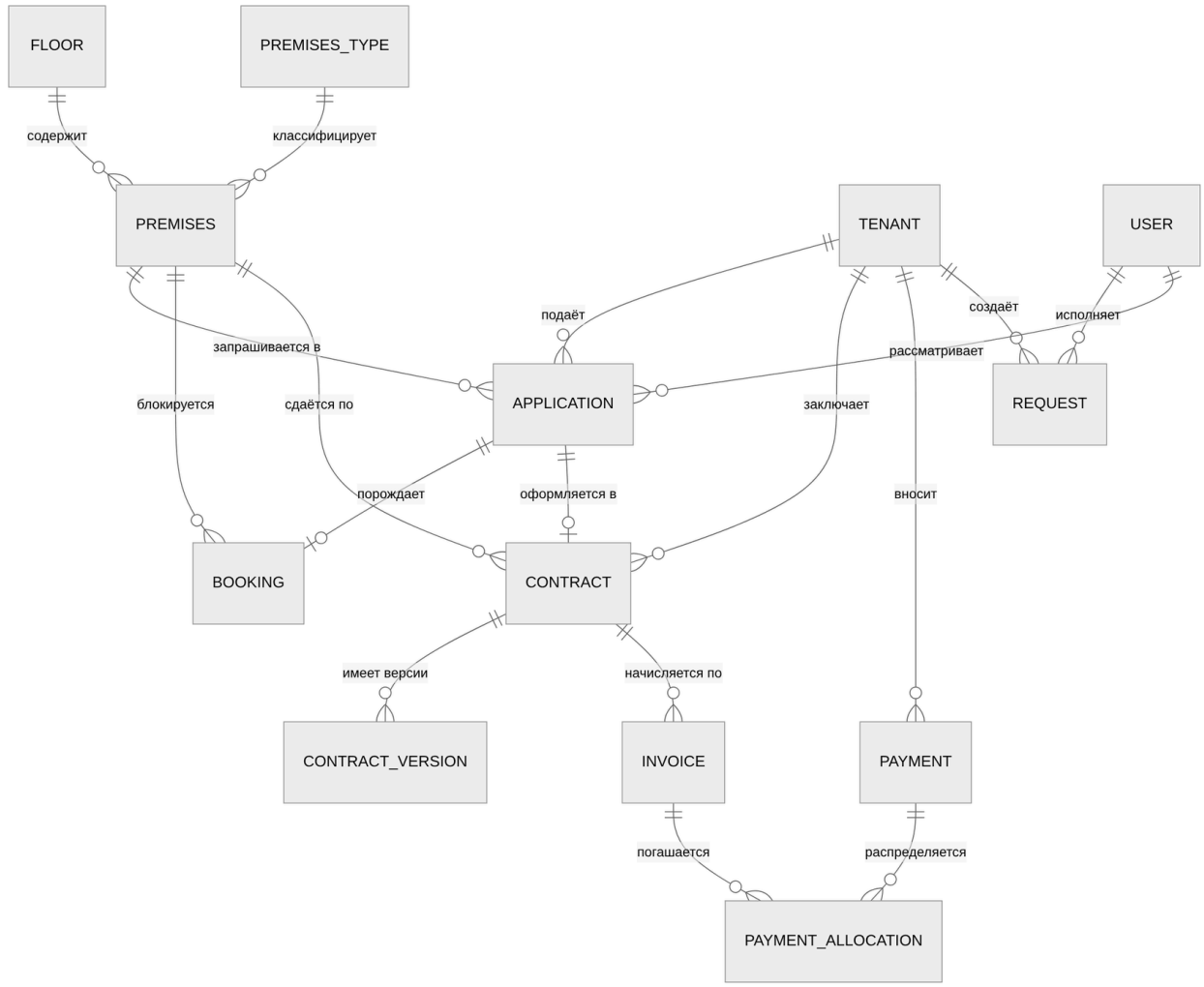


Рисунок 11 — Логическая ER-модель

9.2. Физическая схема (PostgreSQL)

Таблица	Ключевые поля	Ограничения
floor	id PK, number, plan_svg	number UNIQUE
premises_type	id PK, name	name UNIQUE
premises	id PK, floor_id FK, type_id FK, code, area_m2 NUMERIC(8,2), rate_rub_m2 NUMERIC(10,2), status	code UNIQUE; area_m2 > 0; status IN (free, booked, leased, repair)
tenant	id PK, user_id FK, legal_name, inn, ogrn, kpp, bank_details JSONB	inn UNIQUE; длина ИНН 10 или 12

Таблица	Ключевые поля	Ограничения
users	id PK, email, password_hash, role, is_active	email UNIQUE
application	id PK, tenant_id FK, premises_id FK, manager_id FK, period_from, period_to, activity, status, created_at	period_to > period_from
booking	id PK, application_id FK, premises_id FK, valid_from, valid_to, is_active	EXCLUDE USING gist (premises_id WITH =, tstzrange(valid_from, valid_to) WITH &&) WHERE is_active
contract	id PK, number, tenant_id FK, premises_id FK, application_id FK, date_from, date_to, rate_rub_m2, deposit_rub, penalty_pct, status, signed_at	number UNIQUE; EXCLUDE на пересечение периодов по premises_id для статусов signed/active
contract_version	id PK, contract_id FK, version_no, file_url, signature_file_url, created_by FK	(contract_id, version_no) UNIQUE
invoice	id PK, contract_id FK, period_month, rent_amount, service_amount, penalty_amount, total, due_date, status	(contract_id, period_month) UNIQUE
payment	id PK, tenant_id FK, amount, paid_at, bank_doc_no, source	amount > 0
payment_allocation	payment_id FK, invoice_id FK, amount	PK (payment_id, invoice_id)
request	id PK, tenant_id FK, premises_id FK, assignee_id FK, category, text, status, created_at, closed_at	—
audit_log	id PK, user_id FK, entity, entity_id, action, payload JSONB, created_at	только вставка

Ограничения EXCLUDE в таблицах booking и contract реализуют требование целостности: СУБД сама не даст создать две брони или два действующих договора на одно помещение в пересекающиеся периоды.

10. Архитектура системы

Выбрана трёхзвенная клиент-серверная архитектура: SPA-клиент, модульный монолит с REST API и PostgreSQL; фоновые задачи (снятие броней, начисления, напоминания) вынесены в отдельный обработчик очереди.

10.1. Компоненты и развёртывание

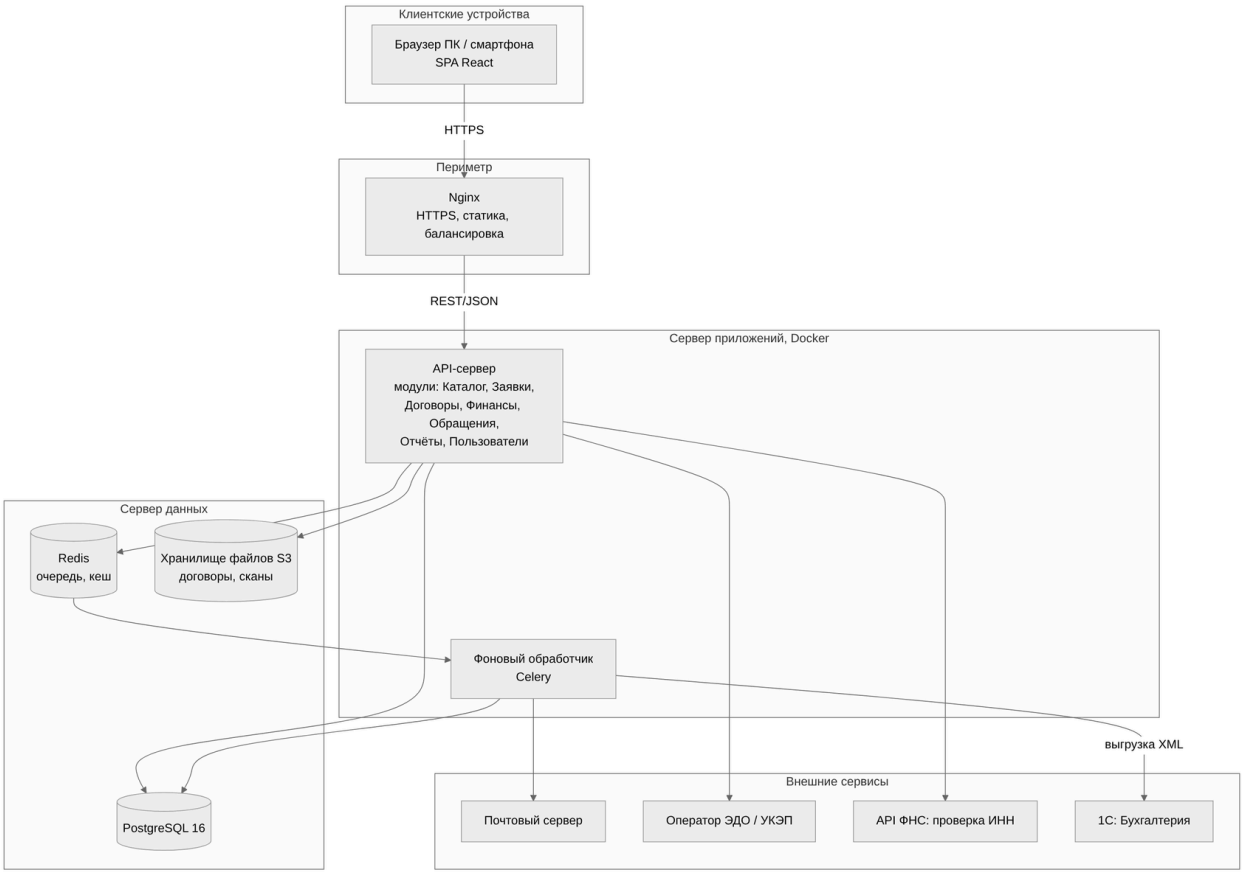


Рисунок 12 — Диаграмма компонентов и развёртывания

10.2. Технологический стек

Слой	Технология	Обоснование
Клиент	React + TypeScript, Ant Design	Готовые компоненты таблиц и форм, типизация
План этажей	SVG + интерактивные области	Помещение кликается прямо на плане, цвет = статус
Сервер	Python 3.12, Django + Django REST Framework	Встроенная админка, ORM, авторизация, быстрая разработка
Фоновые задачи	Celery + Redis	Ежемесячные начисления, снятие просроченных броней, напоминания
СУБД	PostgreSQL 16	Транзакции, EXCLUDE-ограничения на пересечение периодов, JSONB
Документы	docxtpl → PDF (LibreOffice)	Генерация договоров по шаблону Word
Файлы	S3-совместимое хранилище (MinIO)	Версии договоров, подписи, вложения обращений
Инфраструктура	Docker Compose, Nginx	Одинаковое окружение разработки и эксплуатации

10.3. Основные ресурсы REST API

Метод и путь	Назначение
GET /premises?floor=&type=&area_min=&free_from=	Поиск помещений
POST /applications	Подать заявку (создаёт бронь)
POST /applications/{id}/approve, /reject	Решение менеджера
POST /contracts	Сформировать договор из заявки
POST /contracts/{id}/sign	Подписать договор
GET /tenants/me/invoices	Счета арендатора
POST /payments/import	Загрузить банковскую выписку
POST /requests	Создать обращение
GET /reports/occupancy, /reports/debt	Отчёты

11. Проектирование интерфейса

Интерфейс разделён на публичную часть и три рабочих кабинета; центральный элемент — интерактивный план этажа, где цвет помещения показывает его статус.

11.1. Карта экранов

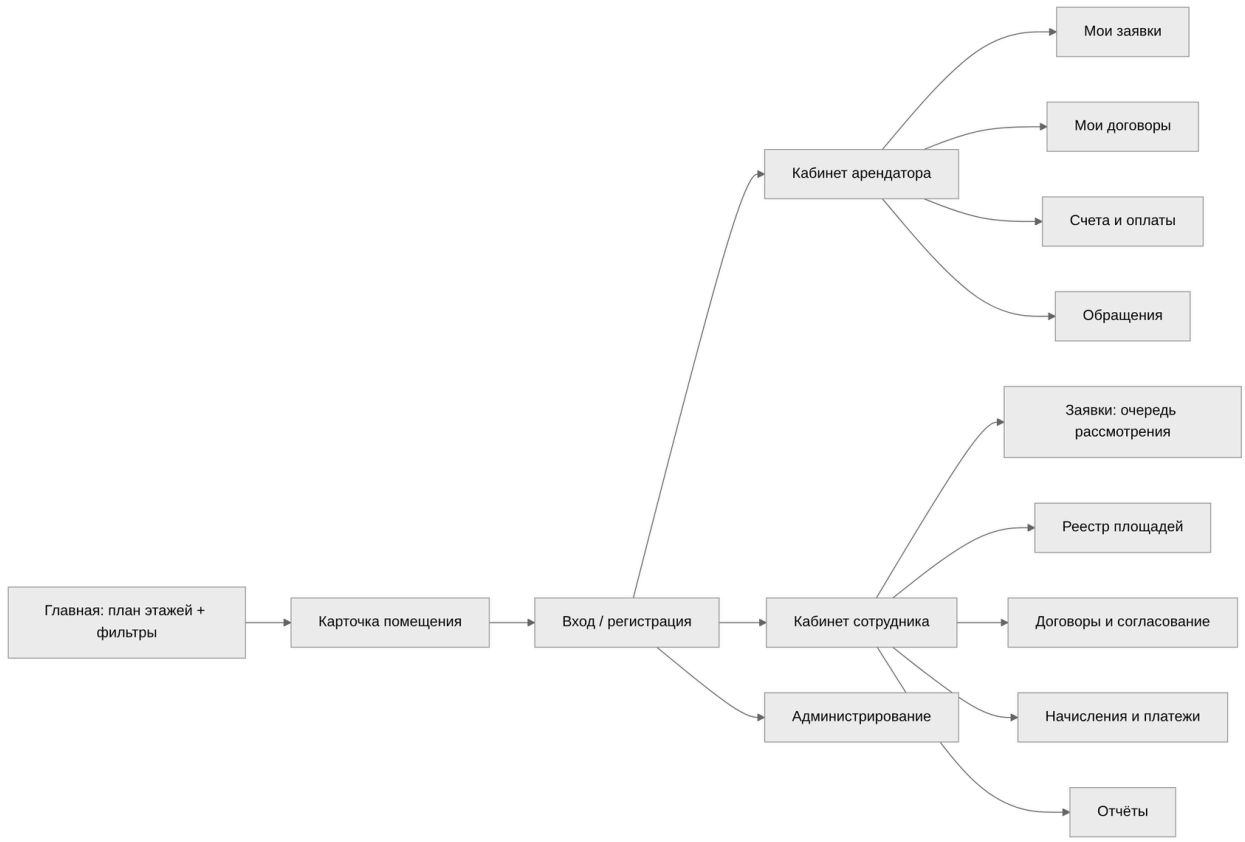


Рисунок 13 — Карта экранов

11.2. Основные экраны

Экран	Пользователь	Содержимое и действия
План этажа	Все	SVG-план, переключатель этажей; зелёный — свободно, жёлтый — бронь, серый — сдано, красный — ремонт; фильтры слева
Карточка помещения	Все	Фото, площадь, ставка, расчёт месячной платы, кнопка «Подать заявку»
Форма заявки	Арендатор	Период аренды (календарь), вид деятельности, предпросмотр стоимости
Кабинет арендатора	Арендатор	Сводка: действующий договор, ближайший счёт, задолженность, открытые обращения
Очередь заявок	Менеджер	Таблица с сортировкой по сроку брони, кнопки «Одобрить» / «Отклонить»
Карточка договора	Менеджер, руководитель	Текст договора, маршрут согласования, история версий, кнопка «Подписать»
Начисления	Бухгалтер	Реестр счетов по месяцу, массовое формирование, загрузка выписки, нераспознанные платежи
Дашборд руководителя	Руководитель	Заполняемость по этажам (%), доход за месяц, топ должников, договоры, истекающие за 60 дней

11.3. Принципы интерфейса

- Не более трёх кликов от главной до подачи заявки.
- Статус любого объекта виден цветным бейджем в списке и карточке.
- Опасные действия (расторжение, отклонение) требуют подтверждения и комментария.
- Адаптивная вёрстка: на смартфоне план этажа заменяется списком с фильтрами.

12. Реализация MVP

По проекту реализован рабочий MVP, который покрывает ключевой путь «каталог → заявка с бронью → договор → подписи → счета → оплаты». Он проверен 28 автотестами и сквозным сценарием в браузере на PostgreSQL 16.

12.1. Стек и объём

Слой	Технология
Сервер	Python 3.12, Django 5.2, Django REST Framework 3.16
База данных	PostgreSQL 16 с расширением btree_gist
Интерфейс	Серверные шаблоны Django, Bootstrap 5.3, SVG-план этажа

Слой	Технология
Документация API	OpenAPI 3 (drf-spectacular), Swagger UI
Фоновые задачи	Процесс `run_scheduler`, запуск раз в минуту
Развёртывание	Docker Compose: db, web (gunicorn), scheduler

Объём кода — около 3 300 строк на Python и 900 строк шаблонов в 7 модулях: accounts, catalog, leasing, finance, core, api и config. Бизнес-логика собрана в `leasing/services.py` и `finance/services.py`. Веб-страницы и REST API вызывают одни и те же функции, поэтому ведут себя одинаково.

12.2. Защита от двойной сдачи помещения

Требование целостности из раздела 5.3 обеспечено двумя независимыми механизмами.

1. **Блокировка строки.** Сервис подачи заявки выполняет `SELECT … FOR UPDATE` по строке помещения, поэтому параллельные заявки на одно помещение обрабатываются по очереди.
2. **Ограничение в СУБД.** Даже при ошибке в коде PostgreSQL не позволит записать пересекающиеся периоды:

```
ExclusionConstraint(
    name="booking_no_overlap",
    expressions=[
        ("premises", RangeOperators.EQUAL),
        (DateRange("lease_from", "lease_to",
RangeBoundary(inclusive_upper=True)),
         RangeOperators.OVERLAPS),
    ],
    condition=Q(is_active=True),
)
```

Такое же ограничение `contract_no_overlap` действует для договоров в статусах «Черновик», «На согласовании», «Ожидает подписи» и «Действует».

12.3. Тестирование

Группа тестов	Что проверяется	Тестов
Ограничения БД	Пересекающиеся брони и договоры отклоняются СУБД в обход сервисов	3
Жизненный цикл аренды	Полный путь, конфликты периодов, отказ, возврат на доработку, истечение брони, недопустимые переходы статусов	10
Финансы	Начисление и его идемпотентность, неполный месяц, импорт выписки с дублями, авансы, пени	5

Группа тестов	Что проверяется	Тестов
Конкурентность	Две одновременные заявки из разных потоков — успешна ровно одна	1
Веб и API	Права ролей, регистрация с проверкой ИНН, полный цикл через REST API с кодом 409 при конфликте	6
Валидаторы и демо-данные	Контрольные суммы ИНН/ОГРН, заполнение базы, регламентные задачи	3

Все 28 тестов проходят. Кроме того, сценарий из пяти ролей пройден в браузере Chromium: заявка, одобрение, договор, утверждение, подпись, загрузка выписки и попытка открыть чужой раздел (HTTP 403).

12.4. Экранные формы

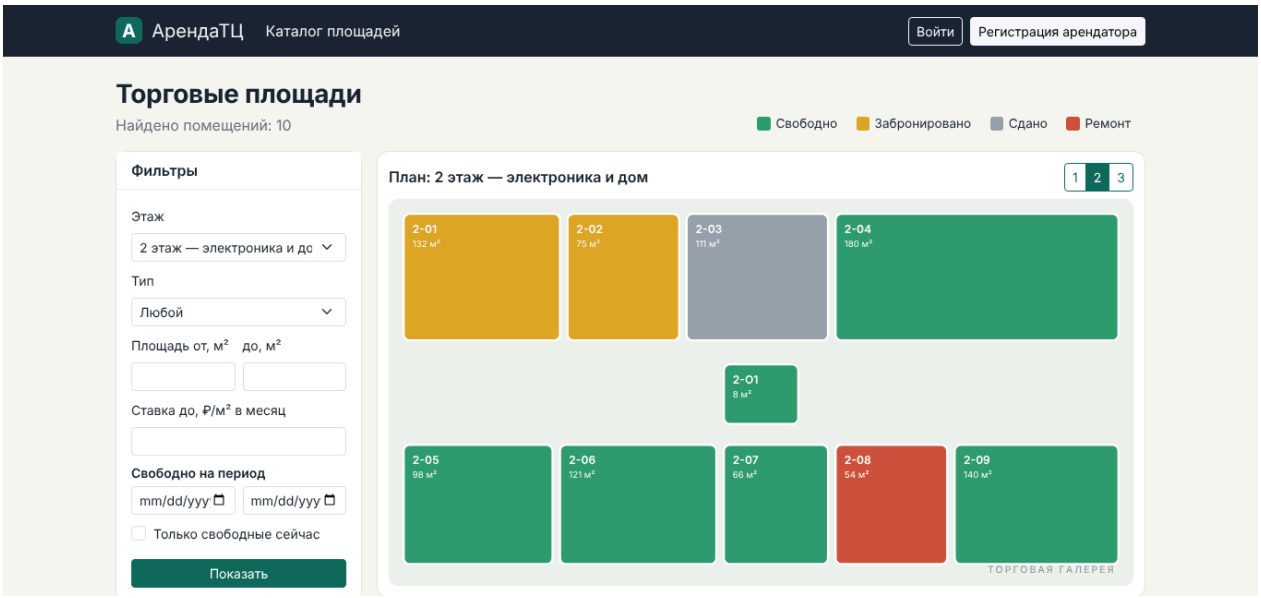


Рисунок 14 — Каталог: цвет помещения на плане показывает его статус.

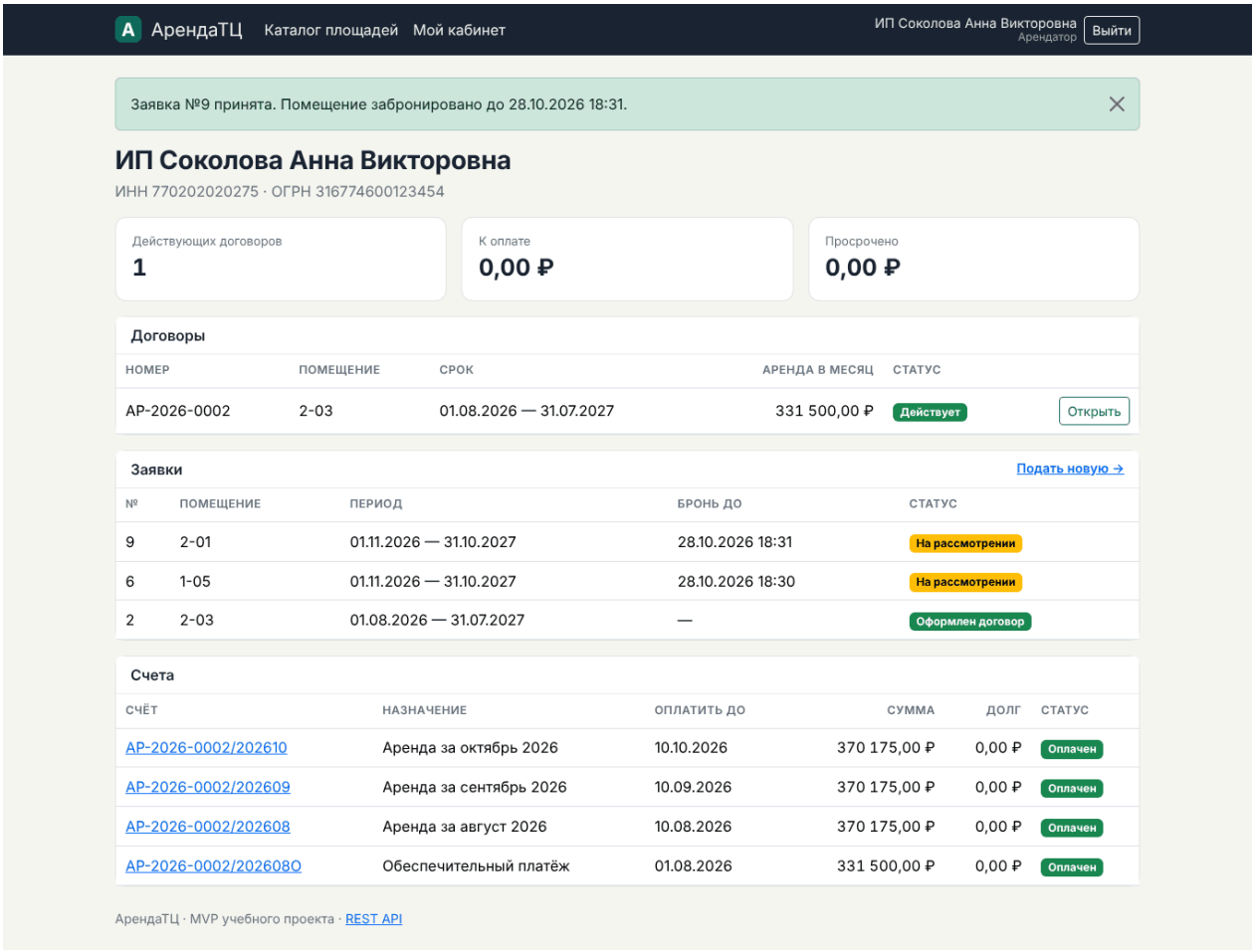


Рисунок 15 — Кабинет арендатора: договоры, заявки, счета и задолженность.

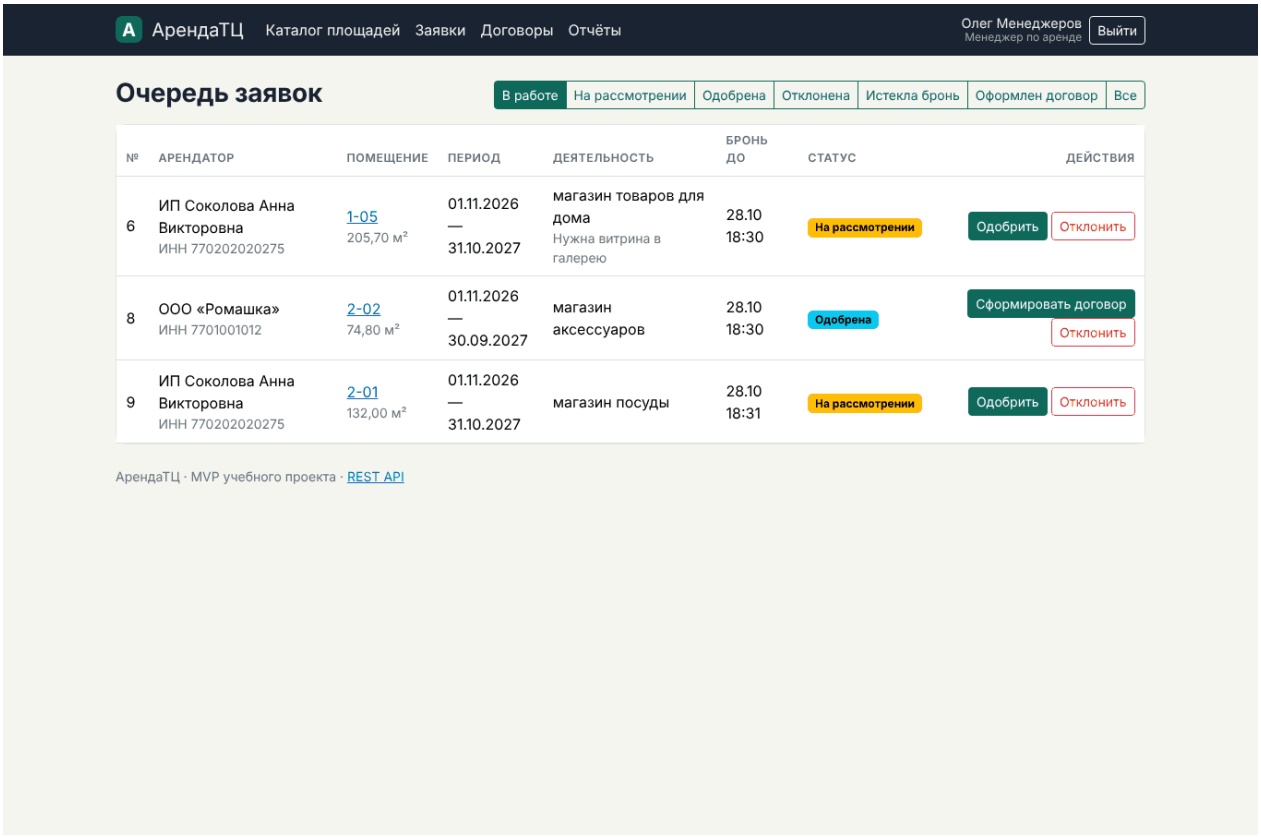


Рисунок 16 — Очередь заявок, отсортированная по сроку окончания брони.

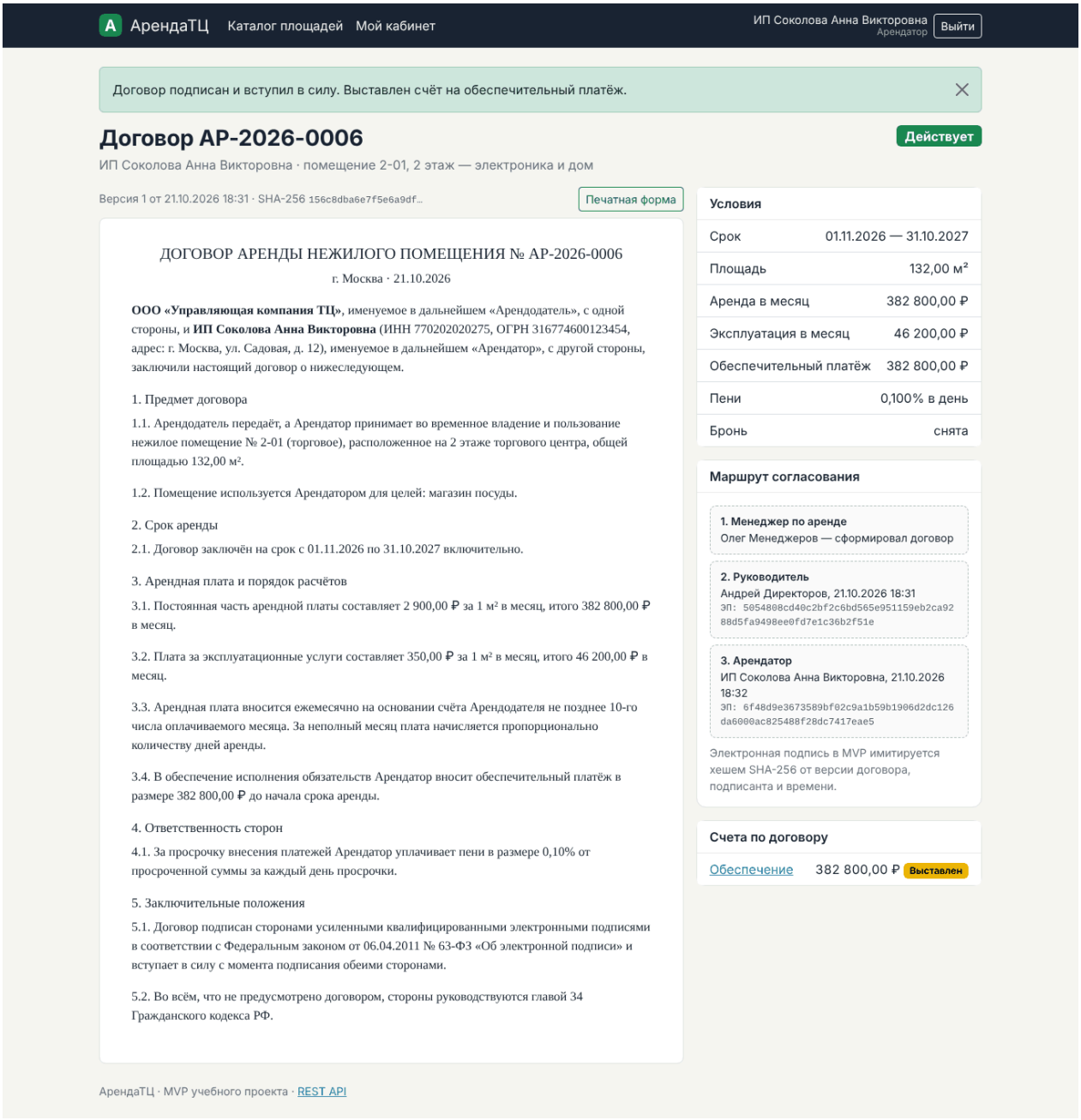


Рисунок 17 — Договор после подписания обеими сторонами: текст версии, условия, маршрут согласования с хешами подписей.

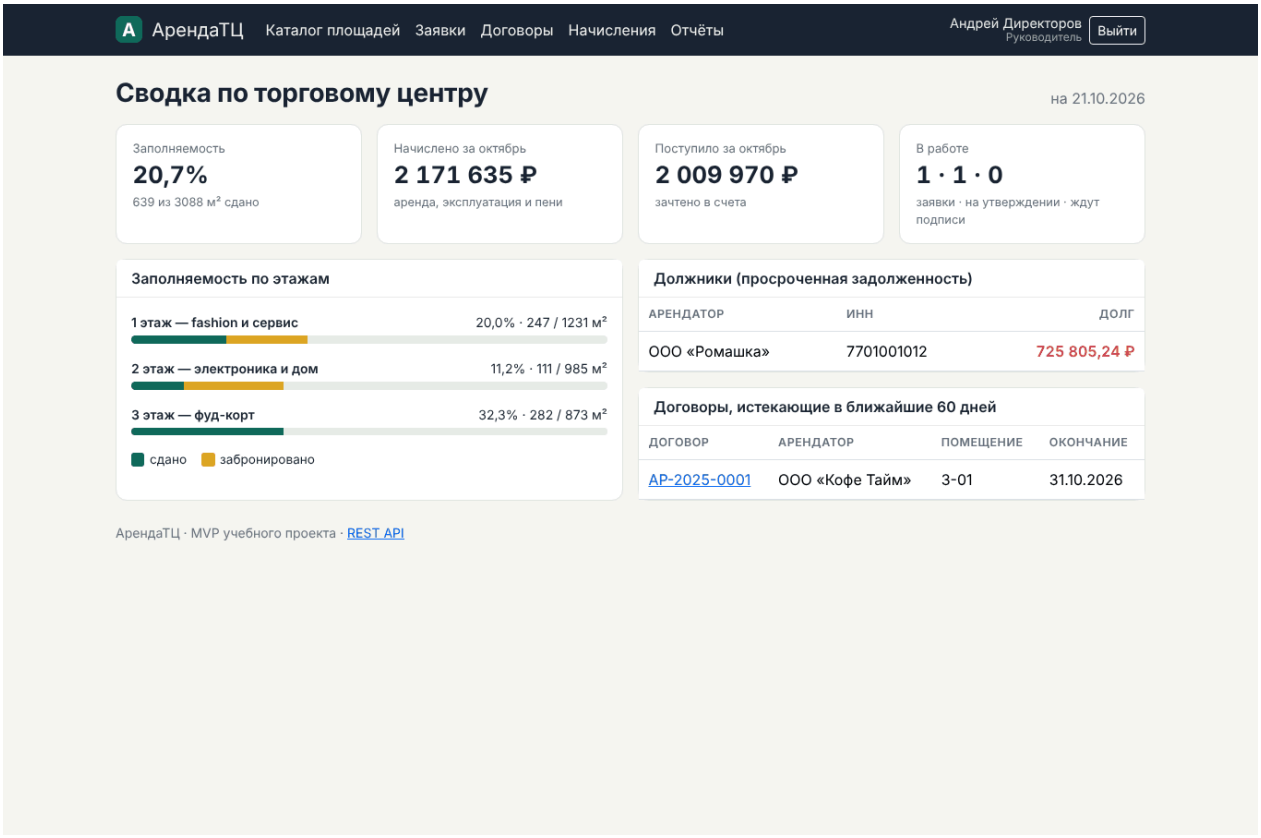


Рисунок 18 — Сводка руководителя: заполняемость, начисления и поступления, должники, истекающие договоры.

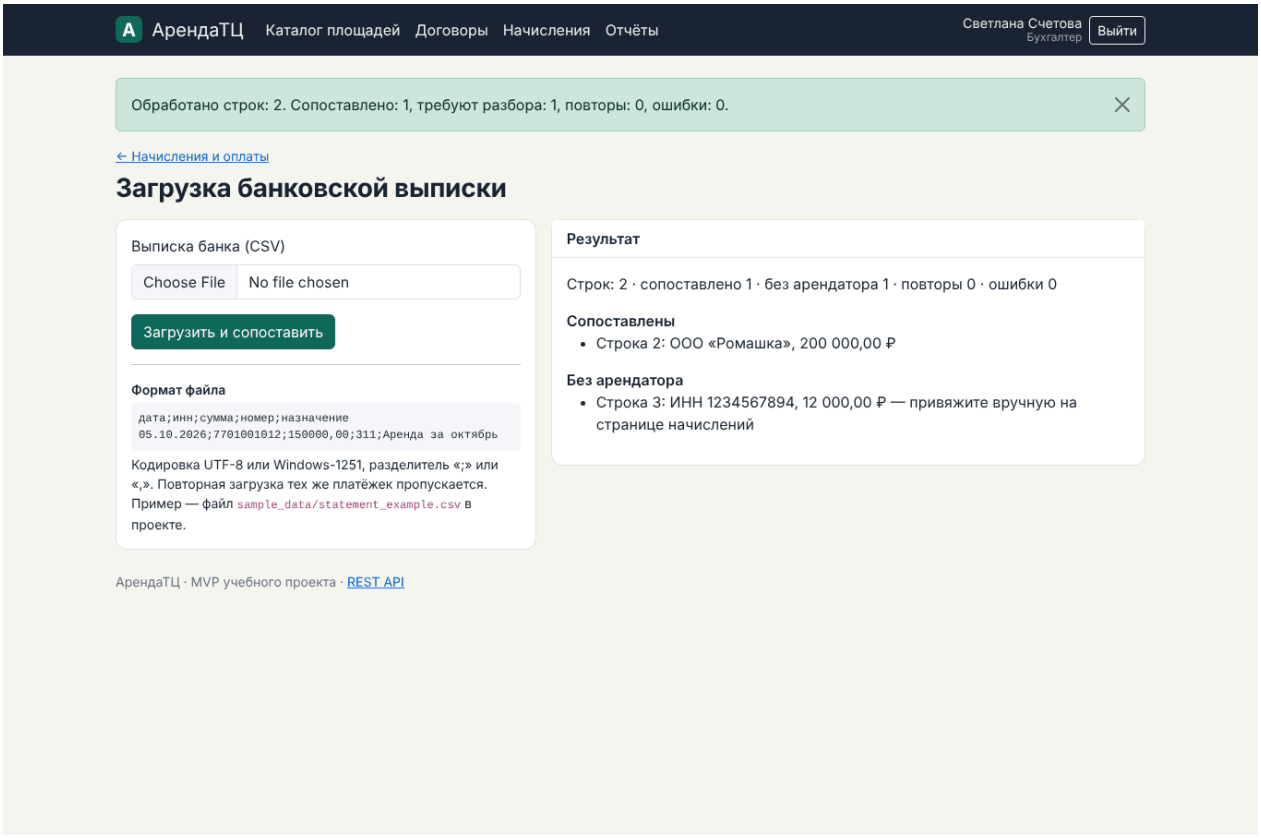


Рисунок 19 — Импорт банковской выписки: один платёж сопоставлен по ИНН, второй отправлен на ручной разбор.

12.5. Отличия MVP от проекта

В проекте	В MVP	Причина
Подпись УКЭП через оператора ЭДО	Хеш SHA-256 от версии договора, подписанта и времени	Нужны договор с оператором и сертификаты; интерфейс функции сохранён
Проверка ИНН через API ФНС	Проверка контрольных сумм ИНН и ОГРН	Нужен ключ доступа к API
Обмен с 1С	Не реализован	Вне ключевого пути
React SPA	Серверные страницы Django	Быстрее для MVP, не нужна сборка фронтенда
Celery + Redis	Один процесс `run_scheduler`	Задачи лёгкие и идемпотентны
Генерация PDF из Word-шаблона	HTML-печатная форма с сохранением в PDF из браузера	Без внешних зависимостей
Обращения арендаторов, отчёт о доходности	Не реализованы	Вне ключевого пути

Запуск — командой `docker compose up --build`. База при первом старте заполняется демо-данными: 3 этажа, 30 помещений, 4 арендатора и 6 договоров с историей счетов.

13. Заключение

В работе спроектирована электронная система сдачи в аренду торговых площадей, покрывающая полный цикл: каталог, заявка и бронь, договор с электронной подписью, начисления и сопоставление оплат, обращения и отчётность.

Полученные результаты:

- функциональная модель IDEF0 (A-0 и декомпозиция A0 на 5 функций) и DFD первого уровня;
- 24 функциональных и 8 групп нефункциональных требований, матрица прав для 6 ролей;
- UML-модели: 16 прецедентов, диаграммы деятельности, последовательности, классов и состояний;
- ER-модель и физическая схема из 14 таблиц PostgreSQL с ограничениями, исключающими двойную сдачу помещения;
- трёхзвенная архитектура на React, Django и PostgreSQL, перечень REST-ресурсов и карта экранов.

По проекту реализован MVP на Django и PostgreSQL. Он покрывает ключевой путь от заявки до учёта оплат, проходит 28 автотестов и запускается одной командой Docker Compose (раздел 12).

Направления развития: мобильное приложение арендатора, онлайн-оплата через эквайринг, переменная арендная плата как процент от оборота арендатора, интеграция с системой учёта посетителей ТЦ.